

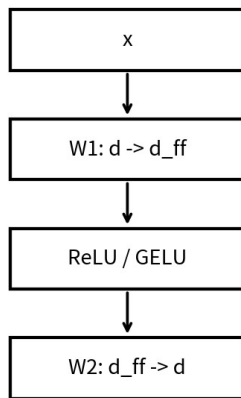
FFN / MLP / SwiGLU

Transformer Feed-Forward Network 深度分析 2026

Position-wise MLP · ReLU/GELU/SiLU · GLU/GEGLU/SwiGLU · 参数/FLOPs · 稀疏化 · 量化 · MoE

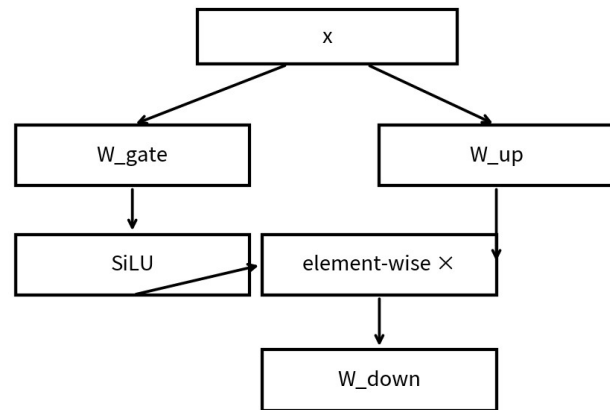
FFN 数据流：从两层 MLP 到门控 SwiGLU

传统 Position-wise FFN



两次大 GEMM + 1 个 activation

SwiGLU FFN



三次大 GEMM + gate/value 乘法

面向：Transformer from scratch、LLM 预训练架构设计、训练/推理 Infra、量化与 Kernel 优化，以及算法/系统面试。

研究基线：Transformer、GELU、GLU、GLU Variants、PaLM、LLaMA、FFN-as-memory、Switch

Transformer、FP8/Smooth-SwiGLU，以及截至 2026-08 的 FFN 稀疏与门控新研究。

版本：2026-08-14。2025-2026 预印本会明确标为“新兴研究”，不与已经广泛验证的工程惯例混为一谈。

0. Executive Summary : FFN 不是“Attention 后面随手接的两层 Linear”

Transformer 的核心 block 可以粗略分成两种运算：Attention 在 sequence 维度做 token-to-token 信息路由；FFN 在每个 token 内部做独立的 channel-wise 非线性变换。后者通常拥有比 attention 更大的参数体量，也是现代 LLM 计算、显存流量、量化和稀疏化优化的主要目标之一。

关键结论：Attention 决定“从哪里读取信息”，FFN 决定“读到以后如何变换、组合和写回 residual stream”。

Transformer Block 的职责分工：Attention 路由，FFN 计算

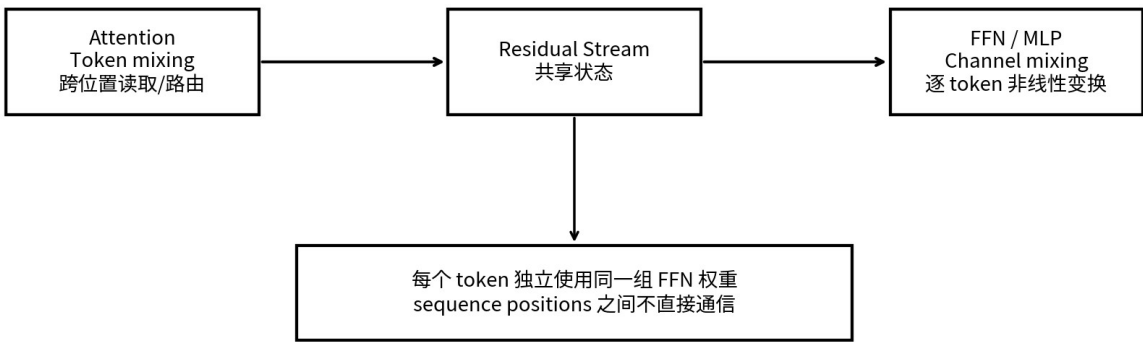


图 1 · Attention 与 FFN 的职责分工

问题	简短答案	容易踩的误区
为什么叫 position-wise?	同一个 MLP 独立作用于每个 sequence position，不直接跨 token 通信	不是“位置编码网络”；position-wise 指逐位置共享参数
为什么先升维再降维?	在更宽的 intermediate feature space 中执行非线性特征组合，再投影回 residual width	4x expansion 不是理论常数
为什么现代 LLM 常用 SwiGLU?	门控 GLU 变体在受控 Transformer 实验中优于 ReLU/GELU，并被 PaLM/LLaMA 等采用	SwiGLU 并非免费：它多一套大矩阵
SwiGLU 为什么 d_int 通常小于 4d?	三矩阵结构要在相近参数/FLOP 预算下缩小 intermediate width	把 4d 原样搬过去会显著增参增算
FFN 是否就是知识库?	FFN-as-key-value-memory 是有价值的解释视角，但不是已证明的唯一机制	不要把单个 neuron 等价成一个事实
为什么 FFN 是 MoE 的自然扩展点?	每个 token 的 FFN 本来就相互独立，可按 token 路由到不同专家	MoE 不是只把一个 dense FFN 复制 N 次这么简单

1. 原始 Transformer 的 Position-wise FFN

1.1 数学定义

原始 Transformer 在每个 encoder/decoder layer 中都放置一个相同结构的两层前馈网络，对每个位置独立应用：

$$\text{FFN}(x) = W2 \cdot \text{ReLU}(W1 \cdot x + b1) + b2$$

$$x \text{ in } \mathbb{R}^{d_{\text{model}}}$$

```
W1: d_model -> d_ff
W2: d_ff -> d_model
```

论文的 base model 使用 $d_{\text{model}}=512$ 、 $d_{\text{ff}}=2048$ ，即经典的 4x expansion ratio。这里的 4 并不是数学必然，而是当时架构/算力预算下的经验选择。

1.2 为什么“同一个 FFN”可以作用于所有 token ？

对输入 $X \in \mathbb{R}^{B \times L \times d}$ ，FFN 可以等价地视为把 $B \cdot L$ 个 token flatten 成二维矩阵后做同一组 GEMM。sequence positions 之间在 FFN 内没有直接耦合，因此 FFN 的计算天然高度并行。

```
X: [B, L, d]
H = act(X @ W1): [B, L, d_ff]
Y = H @ W2:      [B, L, d]
```

关键结论：Position-wise 的“共享”是参数共享，不是激活共享；每个 token 产生自己的 intermediate activation。

2. FFN 为什么先扩维再压回 ？

若只做 $d \rightarrow d$ 的线性层，和前后线性投影叠加后表达能力有限；加入非线性和更宽的中间空间，相当于允许每个 token 激活大量可组合的 feature directions。

视角	$d \rightarrow d_{\text{ff}} \rightarrow d$ 的含义	Know-why
表示容量	把 residual vector 映射到更宽的 feature basis	更宽中间层允许更多非线性分区/特征
字典/记忆视角	W1 产生与一组 learned patterns 的匹配，W2 把被激活 features 写回 residual stream	Geva 等工作将 FFN 分析成 key-value-memory-like 结构
低秩视角	若没有非线性，W2W1 最多是 $\text{rank} \leq d_{\text{ff}}$ 的线性映射	真正价值来自非线性/门控让映射依赖输入
硬件视角	两次大 GEMM 是 GPU/TPU 最擅长的算子	FFN 结构看似简单却非常适合矩阵乘硬件

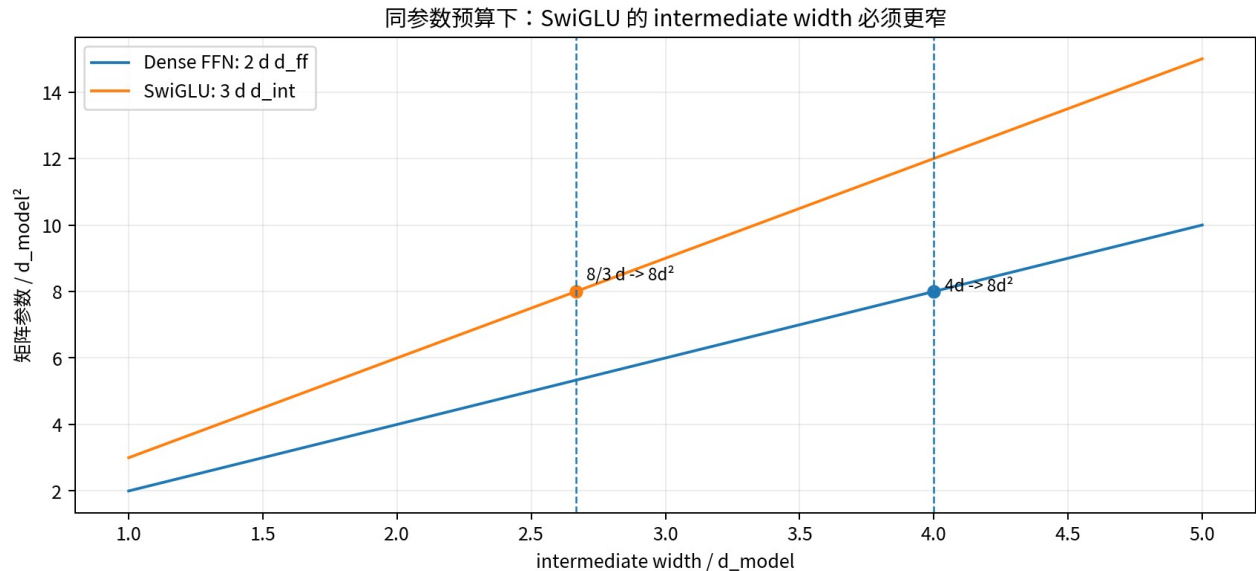


图 2 · intermediate width 直接决定 FFN 的参数/计算预算

3. 参数量与 FLOPs : FFN 为什么常常是模型主体

3.1 传统两层 FFN

忽略 bias，单层 FFN 的矩阵参数近似为：

$$P_{dense} = d * d_{ff} + d_{ff} * d = 2 d d_{ff}$$

若 $d_{ff}=4d$ ，则 $P_{dense} \approx 8d^2$ 。对每个 token，forward 的主计算同样来自两次矩阵乘；若按 multiply-add=2 FLOPs 计，约为 $4 d d_{ff}$ FLOPs。不同论文/工具对 FLOPs 的计数约定可能相差 2 倍，比较时必须统一口径。

3.2 与 Attention 参数比较

标准 MHA 的 Q/K/V/O 投影约为 $4d^2$ 参数；传统 4d FFN 约 $8d^2$ 。因此在不考虑 embedding/LM head 的 block 内，FFN 通常占约 2/3 的线性层参数。这也是“FFN 占大部分参数”的来源之一。GQA 会减少 K/V 投影参数，使 FFN 占比进一步上升。

组件	近似参数量	与 $d_{ff}=4d$ 的比较
MHA QKV+O	$\approx 4d^2$ (标准 MHA)	1.0x baseline
Dense FFN	$2d \cdot 4d=8d^2$	约 attention 的 2x
SwiGLU	$3d \cdot d_{int}$	$d_{int}=8d/3$ 时 $\approx 8d^2$
Embedding/LM head	$V \cdot d$ (是否 tying 决定是否共享)	与 vocab size 强相关

4. 激活函演化 : ReLU -> GELU -> SiLU

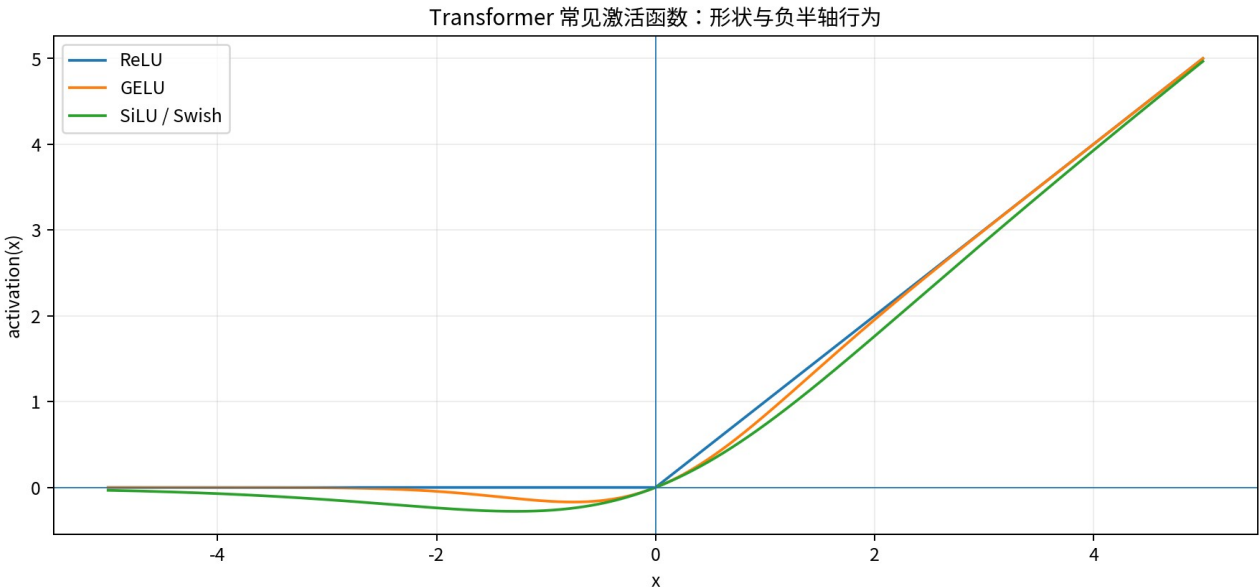


图 3 · ReLU、GELU、SiLU 的输入输出曲线

4.1 ReLU

$$\text{ReLU}(x) = \max(0, x)$$

优势是简单、硬件友好、天然产生大量精确零；缺点是负半轴梯度为 0，边界不光滑。在原始 Transformer 中它是默认 FFN 激活。

4.2 GELU

$$\begin{aligned} \text{GELU}(x) &= x * \text{Phi}(x) \\ \text{Phi}(x) &= \text{standard normal CDF} \end{aligned}$$

GELU 用输入大小连续地决定“通过多少”，而不是像 ReLU 用符号做硬阈值。BERT/GPT 系列广泛使用 GELU。工程实现还存在 exact(erf) 与 tanh approximation 两种常见路径；checkpoint 数值对齐和 kernel 替换时必须确认 approximation 模式。

4.3 SiLU / Swish

$$\text{SiLU}(x) = x * \text{sigmoid}(x)$$

SiLU 同样是平滑且非单调的 self-gated activation。SwiGLU 正是用 SiLU 作为 gate branch 的激活。PyTorch 官方把 SiLU 与 Swish 视作同一函数。

关键结论：平滑激活的主要差异不只是“有没有负值”；它们改变梯度、尾部幅值、激活统计和低精度量化的 outlier 行为。

5. GLU：为什么门控结构与普通激活不同

GLU 最初用于 gated convolutional language model。其核心是两条线性路径逐元素相乘：一条产生内容，另一条产生 gate。

$$\text{GLU}(x) = (x W + b) \odot \text{sigmoid}(x V + c)$$

这个结构和普通“Linear -> activation”有本质差异：非线性不再只对单一路径做 pointwise transform，而是让一组 learned features 条件化地调制另一组 learned features。

结构	公式（省略 bias）	特点
普通 FFN hidden	$\phi(xW)$	单投影后做 pointwise 非线性
GLU	$(xW) \odot \text{sigmoid}(xV)$	$\text{sigmoid gate} \times \text{value}$
ReGLU	$\text{ReLU}(xW_g) \odot (xW_u)$	硬零 gate 更稀疏
GEGLU	$\text{GELU}(xW_g) \odot (xW_u)$	平滑 gate
SwiGLU	$\text{SiLU}(xW_g) \odot (xW_u)$	现代 decoder LLM 常见

6. GLU Variants : SwiGLU 从哪里来？

Shazeer 2020 在 Transformer FFN 中系统比较 GLU 变体，并在其受控实验中发现 GEGLU 与 SwiGLU 获得最好的 perplexity。论文明确表示没有给出决定性的理论解释，因此后续对“为什么 GLU 更好”的机制解释应和这一原始实证结论区分。

关键结论：成熟结论：GLU/SwiGLU 在多种大型 Transformer 中已被反复采用。仍在研究的问题：它究竟通过优化条件数、特征交互还是其他机制带来主要收益。

6.1 SwiGLU 的完整 FFN

```
g = x W_gate
u = x W_up
h = SiLU(g) ⊙ u
y = h W_down
```

$\text{SwiGLU_FFN}(x) = [\text{SiLU}(x W_{\text{gate}}) \odot (x W_{\text{up}})] W_{\text{down}}$

FFN 数据流：从两层 MLP 到门控 SwiGLU

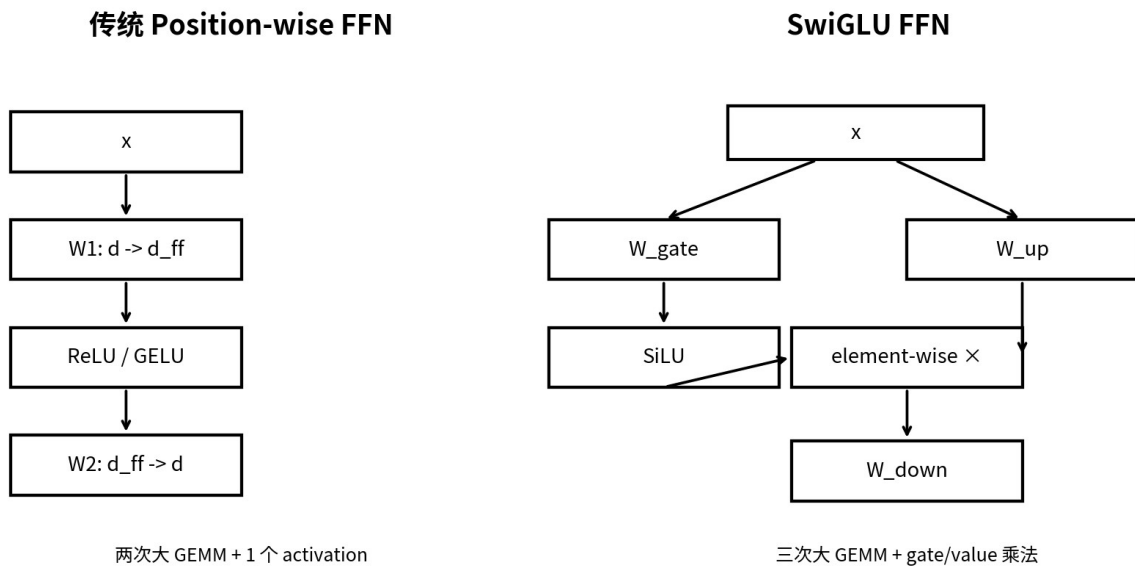


图4 · Dense FFN 与 SwiGLU 的结构差异

6.2 Gate path 与 value path 理解？

- W_{gate} 学习“哪些中间 channels 当前应该被打开/放大/抑制”。
- W_{up} 学习真正要被调制的 value features。
- 逐元素乘法提供二阶/乘性交互，使输出依赖两套不同的线性投影。
- W_{down} 再把 intermediate features 写回 residual stream。

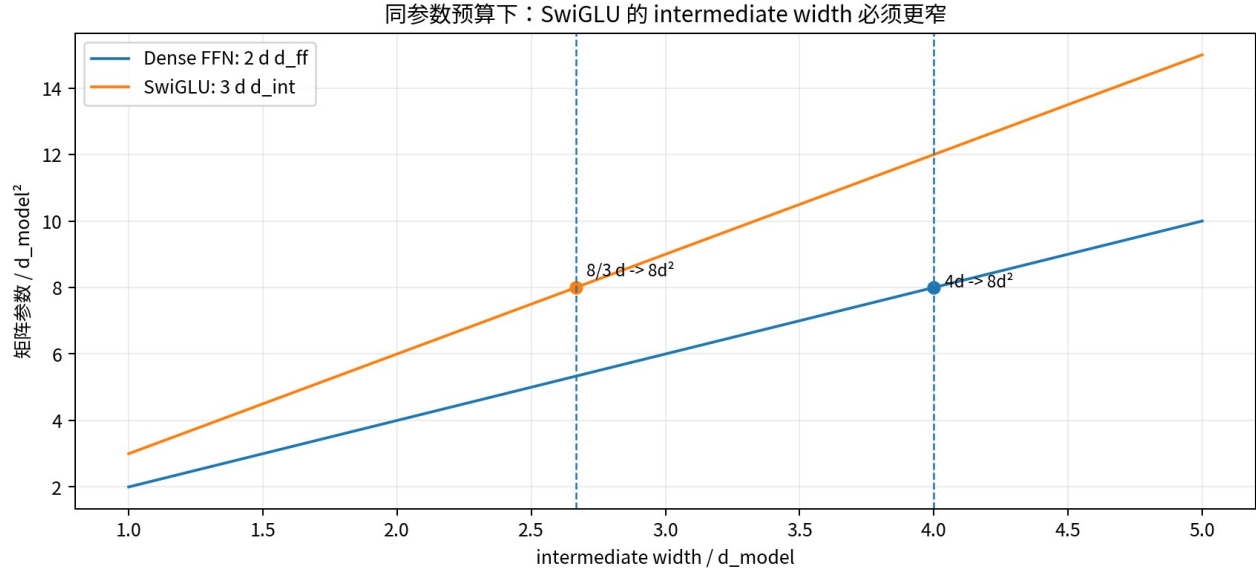
7. 最关键的参数 Gotcha : SwiGLU 的宽度不能照搬 4d

传统 FFN 使用两套大矩阵，SwiGLU 使用三套。若两者 intermediate width 相同，SwiGLU 约多 50% 的主要矩阵参数/计算。

Dense: $P \approx 2 d d_{\text{ff}}$
SwiGLU: $P \approx 3 d d_{\text{int}}$

若 Dense 使用 $d_{\text{ff}}=4d$ ，并希望参数匹配：
 $3 d d_{\text{int}} \approx 2 d (4d)$
 $\Rightarrow d_{\text{int}} \approx 8d/3 \approx 2.667d$

LLaMA 论文明确说明：使用 SwiGLU 后，为匹配参数量把 $4d$ 的 hidden dimension 乘以 $2/3$ ，并再按硬件友好粒度调整。以 $d=4096$ 为例， $8d/3 \approx 10922.7$ ，而 LLaMA-7B 的 intermediate size 使用 11008，体现了“理论比例 + multiple-of rounding”的工程组合。



关键结论：迁移架构时最常见的错误之一，是把传统 `4*d_model` 直接填给 SwiGLU；这样模型大小、FLOPs、optimizer state 和 checkpoint shape 都会一起变化。

8. PaLM / LLaMA 为什么让 SwiGLU 成为主流

PaLM 采用 SwiGLU 作为其 Transformer FFN 激活之一；LLaMA 随后明确采用 SwiGLU 替换 ReLU，并配合 RMSNorm、RoPE、bias-free linear 等现代 decoder 设计。之后 SwiGLU 成为许多开放 LLM 的常见默认 FFN 结构。

架构/论文	FFN 相关选择	值得注意
Transformer 2017	2-layer FFN + ReLU, $d_{ff}=4d$	最经典 baseline
BERT-family	GELU MLP	平滑激活成为主流
PaLM 2022	SwiGLU	把 GLU 变体带入超大规模 dense LM
LLaMA 2023	SwiGLU; intermediate 按参数预算缩放	现代 decoder recipe 的代表
Switch Transformer	把 FFN 替换为按 token 路由的 expert FFN	稀疏条件计算的主线

9. FFN-as-Key-Value Memory：一个有用但不要过度字面化的视角

Geva 等研究发现 Transformer FFN 可以被解释为类似 key-value memory：第一层参数与输入模式匹配，第二层参数对应对词表分布的方向性影响；较低层更偏表面模式，高层更偏语义模式。这个视角帮助解释 FFN 为什么能存储/触发大量特征。

9.1 为什么这个解释有吸引力？

传统 ReLU FFN:
 $h_i = \text{ReLU}(k_i^T x)$
 $y = \sum_i h_i v_i$

可以把每个 intermediate neuron 看成：
 key k_i : 触发条件
 value v_i : 触发后写回 residual 的方向

SwiGLU 使这一分解更复杂，因为每个 intermediate channel 的系数同时依赖 gate 和 value 两条投影，不再是单一 key 的简单 ReLU 匹配。

9.2 重要限制

- “FFN 像 KV memory” 是分析框架，不等于模型事实只存储在 FFN；attention、embedding、residual directions 也共同承载信息。
- 单个 neuron 往往具有 polysemanticity，不能把一个 neuron 机械映射成一个概念。
- 不同 activation、norm、gating 和模型规模下，经典解释未必原样成立。

10. FFN 与 Residual Stream : 真正的写入接口

在 Pre-Norm decoder block 中，典型更新为：

```
h1 = h + Attention(Norm(h))
h2 = h1 + FFN(Norm(h1))
```

因此 FFN 输出不是新的隐藏状态本身，而是对 residual stream 的增量写入。训练中更值得监控的是 FFN update 相对于 residual stream 的尺度，而不仅是 hidden activation 的绝对值。

监控指标	意义	异常可能意味着
ffn_input_rms	进入 FFN 的归一化后尺度	norm/precision 配置异常
gate_rms / up_rms	两条 SwiGLU 分支幅值	某一路径失衡或权重对齐
swiglu_product_rms	乘法后的中间幅值	outlier 放大、FP8 风险
ffn_output_rms	写回 residual 的幅度	过强/过弱 layer update
update/stream ratio	FFN 对当前状态的相对改变量	有效深度下降或训练不稳

11. 初始化 Know-how : 三矩阵门控结构的尺度更敏感

FFN 初始化的目标不是简单让每个 linear 输出方差相同，而是要控制 “gate × value” 后的乘积尺度以及最终 residual update。SwiGLU 的两条 branch 若长期形成相关/对齐，乘积 outlier 可能被放大。

- 不要未经验证地把 Dense FFN 的初始化超参数原样搬到 gated FFN。
- 确认 framework 是否对 W_down 使用 depth-dependent/residual-scaled initialization。
- 观察训练早期和超长训练后的 gate/up correlation 与 activation tail；短跑稳定不代表 trillion-token 稳定。
- 若采用 FP8/MXFP8，activation scaling 与 outlier 监控必须覆盖 SwiGLU product，而不只是 linear 输入。

12. 低精度训练 Gotcha : SwiGLU 会放大 outlier 吗？

2024 的长周期 FP8 训练研究报告：SwiGLU 在数千亿到万亿 token 训练中可能因两分支权重对齐产生持续的 outlier amplification，短训练实验不容易暴露。作者提出 Smooth-SwiGLU 以稳定低精度训练。这个结论对 BF16 并不意味着 SwiGLU 本身 “不稳定”，而是说明门控乘法与低精度 dynamic range 有特殊耦合。

关键结论：数值稳定性测试必须覆盖目标训练 horizon；200M/2B token 稳定，不能自动外推到 2T token。

风险点	原因	工程应对
gate/up 大值相乘	乘法使尾部更重	记录 product amax / percentile；合理 scaling
FP8 scale delayed	历史 amax 跟不上快速变化	更合适的 scaling recipe / block scaling
down projection overflow	product outlier 进入大 GEMM	高精度 accumulate / smooth transformation
optimizer 长周期漂移	branch weight correlation 随时间增长	长跑 proxy 与定期分布监控

13. Activation Sparsity : ReLU 与 SwiGLU 的性能哲学不同

ReLU 会产生精确零，因此天然适合 activation sparsity；GELU/SiLU/SwiGLU 通常不会产生同等程度的硬零。现代稀疏 FFN 研究因此经常尝试预测 “哪些 intermediate channels 重要”，再跳过低贡献计算。

13.1 为什么数学上的稀疏不等于 GPU 加速？

- 非结构化 0 只有在 kernel 能跳过对应 weight tiles 时才有收益。

- 生成 mask 本身有成本；如果先完整算出 dense SwiGLU 才知道 mask，就已经支付主要 GEMM 成本。
- 稀疏矩阵乘需要足够结构化和足够高 sparsity 才能抵消索引/调度开销。
- 不同 batch size、prefill/decode 阶段的瓶颈不同，稀疏收益不可只看理论 FLOPs。

截至 2026 的 Prox 等工作继续研究无需大规模再训练的 SwiGLU channel selection；这类结果仍属于快速发展方向，应以目标硬件的 end-to-end latency 验证，而不是只看 kernel microbenchmark。

14. Dense FFN -> MoE : 为什么专家通常就是 FFN

Mixture-of-Experts 的常见做法不是稀疏化 attention，而是把 dense FFN 替换成多组 expert FFN，并由 router 为每个 token 选择少数专家。原因很自然：FFN 本来就是逐 token 独立模块，替换后可以增加总参数而不让每个 token 激活全部参数。

Dense: $y_t = \text{FFN}(x_t)$
 MoE: $y_t = \sum_{e \in \text{TopK}(\text{router}(x_t))} p_e(x_t) * \text{Expert}_e(x_t)$

Switch Transformer 将 routing 简化到每 token Top-1 expert，并展示了大规模稀疏条件计算的可扩展性。但 MoE 带来新的 load balancing、all-to-all communication、capacity overflow、expert parallelism 和训练稳定性问题，因此应视作 FFN 的下一层系统专题。

关键结论：Dense FFN 的“宽度扩展”增加每 token 计算；MoE 的“专家数扩展”主要增加总参数容量，激活计算可保持相对固定。

15. Tensor Parallel : SwiGLU 为什么特别适合 column/row split

典型 Tensor Parallel MLP 使用：W_gate/W_up 做 column parallel，将 intermediate channels 分片到各 GPU；W_down 做 row parallel，在输出 residual width 上归并。这样大部分 intermediate activation 不需要在设备间全量复制。

```
GPU rank r:
g_r = x @ W_gate[:, shard_r]
u_r = x @ W_up[:, shard_r]
h_r = SiLU(g_r) * u_r
y_partial = h_r @ W_down[shard_r, :]
y = all_reduce(y_partial)
```

Gotcha	错误后果	检查方式
d_int 不能被 TP size 整除	shape / shard mismatch	架构设计时把 multiple_of 与 TP 共同考虑
gate/up shard 顺序不一致	逐元素相乘错 channel	固定 checkpoint layout + parity test
fused QKV 类似地误用到 GLU	split 维度/顺序错误	明确 gate/up 是 2H concat 还是 interleaved
row-parallel all-reduce 时机错误	重复/缺失归并	单卡 vs TP forward parity

16. Fused SwiGLU Kernel : 一个看似简单的 layout 大坑

高性能实现常把 gate 和 up projection 融合成一次更宽的 GEMM，输出 [N, 2H]，随后 fused activation 执行 SiLU(first_half) × second_half。NVIDIA Transformer Engine 文档也明确支持这种 [N,2H] -> [N,H] 的 SwiGLU activation。

```
z = x @ W_gate_up          # [N, 2H]
g, u = split(z, H)
h = silu(g) * u             # [N, H]
y = h @ W_down
```

但不同实现可能采用 contiguous [gate_all, up_all] 或 block-interleaved layout。加载 checkpoint 时只要 layout 假设错了，即使 tensor shape 完全一致，也会得到静默错误。

关键结论：对 fused GLU 来说，“shape 对了”远远不够；必须验证 gate/up 的语义顺序和 interleave 规则。

17. Activation Checkpointing : FFN 中间激活为何昂贵

SwiGLU 训练至少产生 gate、up、activation/product 等 intermediate tensors，其宽度 d_int 通常约 2.67d。对于长 sequence、大 microbatch，这些 activation 占用显著显存。checkpoint/recompute 可以用额外算力换显存。

策略	保存什么	优点	代价
全部保存	gate/up/product	反向最快	显存最高
重算 activation	保存 linear outputs，重算 SiLU/乘法	简单	节省有限
重算 MLP	只保存输入，反向重做两/三次 GEMM	显存大降	计算显著增加
selective recompute	按 kernel/层选择	更灵活	实现复杂

启用 dropout 或 stochastic operations 时，recompute 必须保持 RNG 一致，否则 forward/backward 不再对应同一计算图。很多现代 LLM FFN 本身不使用内部 dropout，但不能据此假设所有架构都没有。

18. Quantization : 为什么 FFN 是 PTQ/QAT 的主战场

FFN 包含模型中大量大矩阵，因此 W4/W8 权重量化、activation quantization 和 mixed precision 大多首先影响 FFN。SwiGLU 的 gate/product 让 activation distribution 比简单 ReLU MLP 更复杂。

量化对象	主要问题	Know-how
W_gate/W_up	不同 channel scale / outlier	分别统计，不要强行共享 scale
W_down	输入来自乘积，尾部可能更重	校准必须覆盖真实 product distribution
activation	SiLU 非零负尾 + gate/value 乘法	per-token/per-channel 或 block scaling 可能更稳
grouped MoE FFN	不同 expert 分布不同	expert-aware calibration / grouped GEMM layout

19. Bias、Dropout 与其他“看似小参数”

设计项	常见现代 LLM 选择	为什么不能盲抄
Linear bias	很多 decoder LLM 设为 False	影响参数、fusion、checkpoint key 与数值
FFN dropout	预训练大模型常设低或 0，但非普遍规律	小数据/微调场景可能需要
activation dropout	T5 等架构可能存在	recompute/RNG 与推理开关要一致
intermediate rounding	multiple_of=128/256 等	需同时考虑 tensor-core tile 和 TP divisibility
activation approximation	GELU exact vs tanh	kernel parity 和导出/部署必须一致

20. 50 ↑ FFN / SwiGLU Gotchas

#	Gotcha	为什么危险
1	把 position-wise 误解成位置编码相关模块	FFN 只是逐 token 独立应用同一 MLP。
2	认为 d_ff=4d 是理论最优	4x 是经典经验点；现代 gated FFN 常用不同比例。
3	SwiGLU 仍用 4d intermediate	会比参数匹配的 dense FFN 多约 50% 主要矩阵参数。
4	只比较 d_ff，不匹配参数/FLOPs	激活函数 ablation 会失去公平性。
5	忽略 multiple_of rounding	最终参数数量和 TP divisibility 会发生变化。
6	fused gate/up 顺序反了	shape 正确但功能完全错误。
7	contiguous 与 interleaved layout 混淆	checkpoint 静默错配。
8	把 W_gate 与 W_up 权重名映射错	模型能跑但 loss/输出异常。
9	认为 SiLU=Sigmoid	SiLU 是 $x * \text{sigmoid}(x)$ 。
10	把 SwiGLU 写成 $\text{SiLU}(\text{gate} * \text{up})$	正确结构是 $\text{SiLU}(\text{gate}) * \text{up}$ 。
11	忘记 W_down	GLU hidden 必须投回 d_model 才能 residual add。
12	GELU exact/tanh approximation 不一致	训练/推理 parity 可能产生小但系统性误差。
13	认为平滑激活一定更稀疏	GELU/SiLU 通常没有 ReLU 的精确零。
14	看到 activation 小就认为可以跳算	硬件必须有稀疏 kernel 才可能省时。
15	先算 dense 再选 sparse mask	可能已经支付主要成本。
16	只看理论 FLOPs 判断 FFN latency	decode 可能受 memory bandwidth、kernel launch 影响。
17	只监控 linear output 不监控 SwiGLU product	乘法后才可能出现最严重 outlier。
18	短跑 FP8 稳定就认为长跑安全	SwiGLU outlier amplification 可能晚期才出现。
19	把 BF16 问题与 FP8 问题混为一谈	dynamic range 和 scaling 机制不同。
20	量化 gate/up 共用不合适的 scale	两分支分布可能不同。
21	忽略 W_down 输入的重尾分布	PTQ calibration 会失真。
22	TP 下 d_int 不可整除	shard shape 直接失败或需要 padding。
23	TP gate/up 分片不一致	逐元素乘法跨了错误 feature。
24	row parallel all-reduce 漏掉/重复	输出幅值错误。
25	把 FFN 参数都当可独立解释 neuron	feature 往往 distributed/polysemantic。
26	把 FFN-as-memory 当作已证明唯一机制	它是解释框架，不是排他的事实存储结论。
27	剪 FFN neuron 只看激活均值	稀有但关键 feature 可能被误删。
28	只以 perplexity 判断稀疏化	long-tail/知识/代码能力可能更敏感。
29	MoE expert width 与 dense width 直接等同	需考虑 activated experts、router、capacity。
30	MoE 只看总参数不看激活参数	推理成本由 activated path 决定。
31	忽略 all-to-all communication	MoE 的系统瓶颈可能从 GEMM 转到网络。
32	expert imbalance 不监控	部分专家拥塞/欠训练。
33	训练时 FFN dropout、推理时配置未正确关闭	输出不确定或不一致。
34	activation checkpoint 忽略 RNG	重算与原 forward 不一致。
35	保存全部 intermediate 导致 OOM	SwiGLU 中间激活很宽。
36	为了省显存全量 recompute 却忽略 wall-clock	GEMM 重算成本很高。
37	FFN bias 配置与 checkpoint 不一致	state_dict mismatch 或隐性行为差异。
38	把模型 hidden_size 与 ff_n_hidden_size 混淆	配置/导出错误。
39	用 vocab projection 的量化策略原样套 FFN	矩阵形状/分布和访问模式不同。
40	只优化 attention kernel	在许多模型中 FFN 仍占大量 block FLOPs。

#	Gotcha	为什么危险
41	prefill/decode 使用同一性能结论	batch/sequence geometry 不同。
42	microbenchmark 快但 end-to-end 不快	融合、通信、调度和显存流量决定整体。
43	改变激活后不重新调 LR/初始化	优化动力学也随之变化。
44	只匹配参数不匹配 FLOPs/token	某些 gated/稀疏结构两者不完全等价。
45	只匹配 FLOPs 不匹配 active capacity	模型能力比较可能不公平。
46	SwiGLU gate/value branch 使用同一个 tensor view 错位	常见手写 split bug。
47	ONNX/TensorRT 导出时激活融合语义变化	需做端到端 parity。
48	checkpoint 转换只检查 tensor shape	layout/语义顺序更重要。
49	把 2026 新 FFN 论文直接当生产共识	需要长期训练、多模型、多硬件复现。
50	改 FFN 后只跑单一 benchmark	应同时看 loss、知识、推理、代码、latency、memory。

21. Know-how : 从零实现一个可靠的 SwiGLU FFN

1. 先固定 d_model、目标 block 参数/FLOPs 预算，再推导 d_int；不要先拍脑袋选 4d。
2. 决定 bias、activation、multiple_of、TP size，并让 d_int 同时满足参数预算和分片整除。
3. 先实现未融合 reference: gate_proj、up_proj、SiLU、逐元素乘、down_proj。
4. 用 FP32 小 tensor 做 shape/数值单元测试，再加入 residual/norm。
5. 实现 fused gate_up projection 后，与 reference 做 max_abs/max_rel parity。
6. 若采用 interleaved layout，写显式 convert 函数与 round-trip test。
7. 加入 TP 后做单卡 vs 多卡输出/梯度 parity。
8. 监控 gate/up/product/output 的 RMS、amax、percentiles 和相关性。
9. 低精度训练必须做长程 proxy，而不是只跑几百 step。
10. 部署前分别 benchmark prefill、decode、batch-size sweep 和真实 sequence-length distribution。

21.1 PyTorch reference MVP

```
class SwiGLUFFN(nn.Module):
    def __init__(self, d_model, d_int, bias=False):
        super().__init__()
        self.gate = nn.Linear(d_model, d_int, bias=bias)
        self.up   = nn.Linear(d_model, d_int, bias=bias)
        self.down = nn.Linear(d_int, d_model, bias=bias)

    def forward(self, x):
        return self.down(F.silu(self.gate(x)) * self.up(x))
```

21.2 参数匹配函数

```
def swiglu_width_for_dense(d_model, dense_ratio=4.0, multiple_of=256):
    # 2*d*(dense_ratio*d) ~= 3*d*d_int
    raw = (2.0/3.0) * dense_ratio * d_model
    return multiple_of * math.ceil(raw / multiple_of)
```

22. Know-why : 20 因果原理

#	Know-why	因果解释
1	为什么 FFN 能逐 token 并行?	它没有 sequence-axis mixing；同一权重独立作用每个 position。
2	为什么需要非线性?	否则 W2W1 可合并成单一线性映射。

#	Know-why	因果解释
3	为什么扩维？	提供更大的 feature basis 与非线性组合空间。
4	为什么 FFN 参数多？	两个/三个 $d \times d_{\text{int}}$ 大矩阵，且 d_{int} 通常是 d 的数倍。
5	为什么 ReLU 会稀疏？	负半轴直接映射为精确 0。
6	为什么 GELU/SiLU 更平滑？	它们以连续 gate 调制输入而非硬阈值。
7	为什么 GLU 比单分支更表达？	两个 learned projections 产生乘性交互和条件门控。
8	为什么 SwiGLU 要缩窄？	多出第三个大矩阵；参数匹配要求 $d_{\text{int}} \approx 8d/3$ 而非 $4d$ 。
9	为什么 d_{int} 要 rounding？	Tensor Core tile、kernel vectorization 和 TP shard 对齐。
10	为什么 FFN 是 MoE 扩展点？	token 独立性允许按 token 选择不同参数子网络。
11	为什么 MoE 能增参不同比增算？	每 token 只激活 Top-K experts。
12	为什么 MoE 仍可能变慢？	routing/all-to-all/负载不均衡带来通信与调度成本。
13	为什么 SwiGLU 低精度更敏感？	两分支大值相乘可扩大 activation tail。
14	为什么 fusion 很重要？	SiLU+multiply+memory traffic 相对 GEMM 虽小，但中间 tensor 读写很昂贵。
15	为什么 FFN 稀疏难落地？	稀疏只有转化成可跳过的结构化 GEMM 才省硬件时间。
16	为什么 W_{down} 很关键？	它决定 activated intermediate features 如何写回 residual basis。
17	为什么 FFN-as-memory 有一定合理性？	激活系数可视为 pattern match，输出列/行提供写回方向。
18	为什么这个 memory 解释不应字面化？	现代模型存在分布式/多义特征，且知识跨模块存储。
19	为什么 FFN 量化收益高？	它包含大量权重和内存流量，是模型成本主体之一。
20	为什么架构 ablation 必须匹配预算？	否则改善可能仅来自更多参数/更多 FLOPs，而非结构本身。

23. Monitoring & Debug Playbook

23.1 训练时建议每层记录

指标	推荐统计	用途
gate preact	RMS, mean, p99, amax	观察 gate scale 与 outlier
up preact	RMS, p99, amax	value path 稳定性
SiLU(gate)	RMS, negative fraction, p99	激活函数分布
product	RMS, p99.9, amax	SwiGLU 乘法尾部
FFN output	RMS, update/stream ratio	写回 residual 的强度
branch correlation	corr(gate, up) 或权重相关指标	长训练对齐/outlier 线索
grad norm	gate/up/down 分开	发现某一路径梯度失衡
update/weight ratio	每矩阵分开	optimizer 有效步长监控

23.2 遇到 NaN / loss spike 的排查顺序

11. 先检查输入 residual/norm 是否已经异常；不要默认是 FFN。
12. 比较 gate、up 和 product 的 amax，确定 outlier 在乘法前还是后出现。
13. 强制该 block 回 BF16/FP32，确认是否为 precision-specific。
14. 关闭 fused kernel，与 reference 实现做 forward/backward parity。
15. 检查 checkpoint layout、gate/up 顺序和 quantization scale。
16. 检查 LR、weight decay、optimizer state，以及异常是否只在特定层/数据 batch 出现。
17. 若只在长训练后发生，分析 branch correlation / weight norm 演化，而不是只复现短窗口。

24. Ablation Matrix : 怎样公平比较 FFN 架构

实验轴	至少比较	必须控制
Activation	ReLU / GELU / SiLU	相同 d_ff、训练 recipe，明确是否只比 activation
Gating	GELU vs GEGLU；SiLU vs SwiGLU	匹配参数和 FLOPs
Width	2d / 8d/3 / 3d / 4d	总参数、训练 token、LR
Bias	on/off	其余完全相同
Precision	BF16 / FP8 recipe	scaling 与 kernel
Sparsity	dense / structured sparse	end-to-end latency，不只理论 FLOPs
MoE	dense vs Top-1/Top-2 experts	active params/FLOPs、总 params、communication
Kernel	reference vs fused	数值 parity + throughput
Layer allocation	uniform width vs layerwise width	总参数固定

评测建议同时包含：held-out LM loss、知识/推理/代码 benchmark、训练吞吐、峰值显存、prefill tokens/s、decode tokens/s、量化误差、长程训练稳定性。

25. 2025-2026 新兴研究：哪些值得跟踪？

方向	代表性现象/工作	当前成熟度判断
为什么 GLU 更好？	2026 条件数/NTK 分析提出 GLU 可能改善谱条件数、加快优化	理论解释正在形成，尚非最终共识
训练后 SwiGLU 稀疏	Prox 等用代理 salience 预测重要 channels	推理研究活跃；需看硬件和模型泛化
新的 GLU 激活	PolyGLU、PowLU 等尝试改变 gate 形状/状态条件化	新兴，缺乏超大规模长期复现
FFN 替代结构	tree-structured / multi-head FFN 等	研究性强，尚未替代标准 SwiGLU
低精度 FFN	Smooth-SwiGLU、MXFP8/NVFP4 kernel	工程价值高，但实现依赖硬件/软件栈

关键结论：生产架构选择应优先参考经过大规模长训练验证的 **recipe**；新激活函数的小模型 **perplexity** 改善不能直接外推到 **100B+** 模型。

26. 面试 / 推导题（25 题）

- 为什么 Transformer 的 FFN 被称为 position-wise？
- 推导传统 $d_{\text{ff}}=4d$ FFN 的参数量。
- 为什么没有 activation 的两层 FFN 可以合并成一层？
- ReLU、GELU、SiLU 的导数/负半轴行为有什么差异？
- GLU 与普通 activation 最大的结构差异是什么？
- 写出 SwiGLU 的完整公式并说明三套矩阵职责。
- 为什么参数匹配的 SwiGLU intermediate width 约为 $8d/3$ ？
- LLaMA-7B $d=4096$ 时 $8d/3$ 是多少？为什么实际会取 11008？
- FFN 和 Attention 在 token mixing/channel mixing 上怎样分工？
- 为什么 FFN 常占 Transformer block 约 2/3 参数？GQA 后会怎样？
- FFN-as-key-value-memory 如何从 ReLU MLP 公式推导？
- 为什么该解释不应被理解为“一个 neuron=一个事实”？
- SwiGLU 为什么可能产生低精度 outlier amplification？
- 设计一个监控 SwiGLU 长训练稳定性的指标集合。
- fused gate/up projection 有哪些 checkpoint layout gotchas？
- Tensor Parallel 下 gate/up/down 应如何分片？
- 为什么 d_{int} 要考虑 TP divisibility？
- activation sparsity 为什么不自动转换成 latency speedup？
- Dense FFN 转 MoE 后，总参数、active params、FLOPs 分别怎么变化？
- 为什么 MoE 的主要系统代价是 all-to-all？
- 比较 activation checkpointing 与 selective recompute 在 FFN 上的代价。
- 量化 SwiGLU 时为什么 product distribution 很关键？
- 如何公平 ablate GELU MLP vs SwiGLU？
- 如果新 SwiGLU kernel 比旧 kernel 快 20%，为什么 end-to-end 可能只快 5%？
- 如果让你设计下一代 FFN，你会同时优化哪些算法指标与系统指标？

27. 一页速查表

主题	公式 / 经验	记忆点
Dense FFN	$W_2 \phi(W_1 x)$	两矩阵；经典 $d_{ff}=4d$
GELU	$x \Phi(x)$	平滑概率门控风格
SiLU	$x \text{sigmoid}(x)$	Swish
GLU	$(xW) \odot \text{sigmoid}(xV)$	$\text{gate} \times \text{value}$
SwiGLU	$[\text{SiLU}(xW_g) \odot xW_u]W_d$	三矩阵现代 LLM 常见
参数匹配	$d_{int} \approx 8d/3$	对应传统 4d dense FFN
Dense 参数	$2 d d_{ff}$	忽略 bias
SwiGLU 参数	$3 d d_{int}$	忽略 bias
TP	gate/up column-parallel; down row-parallel	中间 features 本地保留
低精度	重点监控 product tail	长训练可能才暴露
MoE	router + 多 expert FFN	增加总容量，稀疏激活

28. 参考文献与核验来源

[1] Vaswani et al. Attention Is All You Need. arXiv:1706.03762, 2017.

[2] Hendrycks & Gimpel. Gaussian Error Linear Units (GELUs). arXiv:1606.08415, 2016.

[3] Dauphin et al. Language Modeling with Gated Convolutional Networks. arXiv:1612.08083, 2016/2017.

[4] Shazeer. GLU Variants Improve Transformer. arXiv:2002.05202, 2020.

[5] Chowdhery et al. PaLM: Scaling Language Modeling with Pathways. arXiv:2204.02311, 2022.

[6] Touvron et al. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971, 2023.

[7] Geva et al. Transformer Feed-Forward Layers Are Key-Value Memories. arXiv:2012.14913 / EMNLP 2021.

[8] Fedus, Zoph, Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. arXiv:2101.03961.

[9] Fishman et al. Scaling FP8 Training to Trillion-Token LLMs. arXiv:2409.12517, 2024.

[10] PyTorch Documentation. torch.nn.GELU / torch.nn.SiLU. Accessed 2026-08.

[11] NVIDIA Transformer Engine Documentation. GLU/SwiGLU activation and fused/interleaved layouts. Accessed 2026-08.

[12] Lyu et al. The Devil is in the Condition Numbers: Why is GLU Better than non-GLU Structure? arXiv:2605.20749, 2026. [新兴理论]

[13] Liu et al. Prox: Training-Free FFN Activation Sparsity via Approximate Intermediate-Channel Saliency in LLMs. arXiv:2607.27591, 2026. [新兴研究]

说明：本报告中的参数/FLOPs/shape 公式均由上述架构定义直接推导；不同框架对 FLOPs 的 multiply-add 计数可能采用不同约定，进行横向比较时应统一口径。